

robo forge Bericht

Hund

Ein Hund-Roboter der auf Arduino Mega basiert, viele lichter hat sowie eine kamera und vernsteuerung außerdem hat es einen oled display als „gesicht“ des hundes

Dokumentation, Komponenten, Verdrahtung, Code, 3D-Modell und Prüfliste.

Projektspezifikation

Projekt Spezifikation: Hund-Roboter

Ziel

Das Projekt zielt darauf ab, einen einfachen, funktionsfähigen Roboter zu entwickeln, der als Hund modelliert ist. Der Roboter soll grundlegende Bewegungen, Lichteffekte, eine Kamera und eine Anzeige aufweisen, um die Funktionalität eines Haustiers im robotischen Kontext zu simulieren.

Robotertyp

Der Roboter ist ein beweglicher, interaktiver Haustier-ähnlicher Roboter. Er ist auf einer einfachen Plattform aufgebaut, die Bewegung und Lichtsteuerung ermöglicht. Die Gestaltung ist an einen Hund angelehnt, wobei die Hauptfunktionen auf die Interaktion mit dem Benutzer ausgerichtet sind.

Hauptfunktionen

- Bewegung (z. B. Vorwärts, Rückwärts, Drehung)
- Lichteffekte (LEDs)
- Kamera (z. B. zur Erkennung von Bewegungen oder zur Aufnahme von Bildern)
- OLED-Display als „Gesicht“ des Roboters
- Steuerung über Arduino Mega

Vermutete Architektur

- Zentrale Steuerung durch Arduino Mega
- Hardware-Module für Bewegung, Licht, Kamera und Display sind über I²C oder SPI angeschlossen
- Softwaremodul zur Steuerung der Module und zur Verarbeitung von Eingaben
- Eingabemöglichkeiten über Tasten, Sensoren oder Fernbedienung

Controller

- Arduino Mega (ATmega2560)
- Verwendung von Libraries zur Steuerung von LEDs, Kamera, Display und Motoren

Energieversorgung

- Stromversorgung über Akkus (z. B. 12 V LiPo oder AA-Batterien)
- Stromversorgung für verschiedene Module über Spannungsregler (z. B. 5 V und 3.3 V)
- Akku-Ladefunktion oder Austauschmöglichkeit

Sensorik

- Bewegungssensor (z. B. PIR)
- Kamera (z. B. OV7670 oder ähnlich)
- Taster oder Knöpfe zur manuellen Steuerung
- Eventuell Temperatur- oder Lichtsensor zur Umgebungswahrnehmung

Aktoren

- Motoren zur Bewegung (z. B. zwei DC-Motoren mit Treiber)

- LEDs zur Beleuchtung
- OLED-Display (z. B. 128x64 Pixel, I²C-Anschluss)
- Eventuell Servos zur Bewegung von Gliedmaßen

Risiken

- Kompatibilitätsprobleme bei der Ansteuerung von Sensoren und Aktoren
- Stromverbrauch kann die Akkulaufzeit beeinträchtigen
- Softwarefehler bei der Kommunikation zwischen Modulen
- Hardwaredefekte durch falsche Verkabelung oder Überlastung
- Unzureichende Leistung der Kamera oder des Displays

MVP-Grenzen

- Die Bewegung ist auf einfache Vorwärts-/Rückwärts-Bewegungen begrenzt
- Die Kamera wird nur für grundlegende Funktionen genutzt (keine fortschrittliche Bildverarbeitung)
- Die Lichteffekte sind auf statische LEDs begrenzt
- Das OLED-Display zeigt nur einfache Grafiken oder Texte an
- Die Steuerung ist nicht vollständig autonom, sondern erfordert manuelle Eingriffe

Dieses Dokument beschreibt die geplanten Funktionen und technischen Annahmen des Projekts. Es dient als Grundlage für die Projektplanung und Präsentation.

Hinweis: Hinweis: Dieser Inhalt wurde lokal mit RoboForge Local erzeugt. Er ist eine Lern- und Entwurfsvorlage und muss vor realer Hardware, Druck oder Abgabe geprüft werden.

Projektübersicht

Hund – Roboterprojekt

Ziel

Das Projekt zielt darauf ab, einen einfachen, funktionsfähigen Roboter zu entwickeln, der als Hund modelliert ist. Der Roboter soll grundlegende Bewegungen, Lichteffekte, eine Kamera und eine Anzeige aufweisen, um die Funktionalität eines Haustiers im robotischen Kontext zu simulieren. Ziel ist es, ein interaktives Lernprojekt zu erstellen, das sowohl technisches Verständnis als auch kreatives Gestalten fördert.

Funktionsidee

Der „Hund“-Roboter ist ein beweglicher, interaktiver Haustier-ähnlicher Roboter, der auf einer einfachen Plattform aufgebaut ist. Er verfügt über:

- Bewegung: Vorwärts, Rückwärts, Drehung
- Lichteffekte: LEDs zur Beleuchtung und Signalisierung
- Kamera: Zur Erkennung von Bewegungen oder Aufnahme von Bildern
- OLED-Display: Als „Gesicht“ des Roboters
- Steuerung: Über Arduino Mega

Die Gestaltung ist an einen Hund angelehnt, wobei die Hauptfunktionen auf die Interaktion mit dem Benutzer ausgerichtet sind.

Projektstruktur

Das Projekt besteht aus folgenden Hauptkomponenten:

- Hardware:
 - Arduino Mega als zentrale Steuerung
 - LEDs, Motoren, Kamera, OLED-Display
 - Sensoren (z. B. PIR, Taster)
 - Stromversorgung (Akku, Spannungsregler)
- Software:
 - Steuerung der Module über Arduino-Code
 - Kommunikation über I²C oder SPI
 - Eingabeverarbeitung (Tasten, Sensoren)

Arbeitsablauf

1. Planung: Festlegung der Funktionen und Komponenten
2. Hardware-Baugruppe: Aufbau der elektrischen Schaltung und Montage der Module
3. Software-Entwicklung: Programmierung der Steuerung und Funktionen
4. Integration: Zusammenfügen von Hardware und Software
5. Testen: Überprüfung der Funktionalität und Anpassung

MVP-Grenzen

Das Projekt ist als Minimum Viable Product (MVP) konzipiert und hat folgende Einschränkungen:

- Bewegung ist auf einfache Vorwärts-/Rückwärts-Bewegungen begrenzt
- Kamera wird nur für grundlegende Funktionen genutzt (keine fortschrittliche Bildverarbeitung)
- Lichteffekte sind auf statische LEDs begrenzt
- OLED-Display zeigt nur einfache Grafiken oder Texte an
- Steuerung ist nicht vollständig autonom, sondern erfordert manuelle Eingriffe

Sicherheitshinweis

Beim Umgang mit elektrischen Komponenten und Akkus ist auf Sicherheit zu achten. Es wird empfohlen, immer eine ordnungsgemäße Verkabelung und Stromversorgung zu verwenden. Bei der Montage ist auf Kurzschlüsse und Überlastungen zu achten. Das Projekt ist für Bildungszwecke gedacht und soll unter Aufsicht durchgeführt werden.

Dieses Projekt dient der technischen und kreativen Weiterentwicklung von Robotik-Konzepten und ist kein fertiges Industrieprodukt.

Hinweis: Hinweis: Dieser Inhalt wurde lokal mit RoboForge Local erzeugt. Er ist eine Lern- und Entwurfsvorlage und muss vor realer Hardware, Druck oder Abgabe geprüft werden.

Komponenten

Komponenten

Controller

Arduino Mega (ATmega2560)

Zweck: Zentrale Steuerung des Roboters mit ausreichend I/O-Pins für diverse Module.

Warum passend: Die Mega bietet mehr Pins als kleinere Controller wie Uno, was für die Ansteuerung von LEDs, Motoren, Kamera und Display erforderlich ist.

Zu prüfen: Kompatibilität der verwendeten Libraries mit der Mega-Plattform.

Hinweis: Für die Kommunikation mit Sensoren und Aktoren über I²C/SPI muss die korrekte Pinbelegung berücksichtigt werden.

Energieversorgung

LiPo-Akku (12 V)

Zweck: Stromversorgung des gesamten Roboters.

Warum passend: Hohe Energiekapazität und geringes Gewicht für tragbare Anwendungen.

Zu prüfen: Passende Spannungsregler (z. B. 5 V und 3.3 V) zur Versorgung der Module.

Hinweis: Akku-Ladefunktion oder Austauschmöglichkeit sollte im Design berücksichtigt werden.

Sensorik/Kamera

Kamera (OV7670 oder ähnlich)

Zweck: Erkennung von Bewegungen und Aufnahme von Bildern.

Warum passend: Geringe Kosten, einfache Ansteuerung über I²C, geeignet für grundlegende Bildverarbeitung.

Zu prüfen: Kompatibilität mit Arduino Mega und benötigte Speicherkapazität.

Hinweis: Die Kamera wird nur für grundlegende Funktionen genutzt, keine fortschrittliche Bildverarbeitung.

Motoren/Aktoren

DC-Motoren (zwei)

Zweck: Bewegung des Roboters.

Warum passend: Einfache Ansteuerung über Treiber (z. B. L298N), geeignet für einfache Bewegung.

Zu prüfen: Passende Treiber für die benötigte Stromstärke.

Hinweis: Die Bewegung ist auf Vorwärts-/Rückwärts-Bewegungen begrenzt.

Motoren/Aktoren

LEDs (zahlreiche)

Zweck: Lichteffekte zur Visualisierung des Roboters.

Warum passend: Einfache Ansteuerung über digitale Pins, viele Farben und Effekte möglich.

Zu prüfen: Stromaufnahme und Passung zur Spannungsversorgung.

Hinweis: LEDs sind statisch, keine dynamischen Effekte wie PWM oder Farbwechsel.

Motoren/Aktoren

OLED-Display (128x64 Pixel, I²C)

Zweck: Anzeige als „Gesicht“ des Roboters.

Warum passend: Kleine Größe, hohe Auflösung, einfache Ansteuerung über I²C.

Zu prüfen: Kompatibilität mit Arduino Mega und verwendete Bibliothek.

Hinweis: Anzeige ist auf einfache Grafiken und Texte begrenzt.

Gehäuse/Befestigung

Roboter-Gehäuse (3D-gedruckt oder aus Holz)

Zweck: Struktur und Form des Hundes.

Warum passend: Einfache Gestaltung, Anpassung an die benötigten Module.

Zu prüfen: Passform der Module im Gehäuse, Stabilität und Schutz.

Hinweis: Das Gehäuse sollte leicht zu reparieren und zu erweitern sein.

Zusatzteile

Taster/Knöpfe

Zweck: Manuelle Steuerung des Roboters.

Warum passend: Einfache und zuverlässige Eingabemöglichkeit.

Zu prüfen: Anzahl und Position der Taster im Gehäuse.

Hinweis: Taster können zur Steuerung von Funktionen wie Licht, Bewegung oder Display genutzt werden.

Zusatzteile

PIR-Bewegungssensor

Zweck: Erkennung von Bewegungen in der Umgebung.

Warum passend: Einfache Ansteuerung, geringe Kosten, gut für grundlegende Interaktion.

Zu prüfen: Sensorposition und Reichweite im Gehäuse.

Hinweis: Sensor wird zur Aktivierung oder Steuerung des Roboters genutzt.

Zusatzteile

Spannungsregler (5 V und 3.3 V)

Zweck: Versorgung der Module mit passender Spannung.

Warum passend: Ermöglicht gleichzeitige Nutzung verschiedener Module mit unterschiedlichen Spannungen.

Zu prüfen: Leistungsfähigkeit des Reglers für die benötigte Stromstärke.

Hinweis: Regler sollten über Wärmeableitung verfügen, um Überlastung zu vermeiden.

Hinweis: Hinweis: Dieser Inhalt wurde lokal mit RoboForge Local erzeugt. Er ist eine Lern- und Entwurfsvorlage und muss vor realer Hardware, Druck oder Abgabe geprüft werden.

Verdrahtung

Wiring.md – Hund-Roboter

Projektübersicht

Das Projekt „Hund“ ist ein interaktiver Roboter, der auf einem Arduino Mega basiert. Er simuliert ein Haustier mit Bewegung, Lichteffekten, Kamera und einem OLED-Display als „Gesicht“. Die Hardware ist modular aufgebaut, wobei alle Module über I²C oder SPI an den Controller angeschlossen sind. Die Steuerung erfolgt über ein Arduino Mega (ATmega2560), das alle Module steuert und Eingaben verarbeitet.

Hardware-Komponenten und Anschlüsse

1. Arduino Mega (ATmega2560)

- Controller: Arduino Mega (ATmega2560)
- Versorgung: 12V Akku über Spannungsregler (z. B. LM7805 für 5V, LM317 für 3.3V)
- Anschlüsse:
- I²C: SDA (Pin 20), SCL (Pin 21)
- SPI: MISO (Pin 50), MOSI (Pin 51), SCK (Pin 52), SS (Pin 53)
- PWM-Pins für Motoren und LEDs
- Serielle Schnittstellen (UART) für Debugging

2. OLED-Display (128x64 Pixel, I²C)

- Typ: SSD1306 (I²C-Anschluss)
- Versorgung: 3.3V (über Spannungsregler)
- Anschlüsse:
- VCC → 3.3V
- GND → gemeinsame Masse
- SDA → Pin 20 (I²C)
- SCL → Pin 21 (I²C)

3. Kamera (OV7670)

- Typ: OV7670 (I²C- und Parallel-Schnittstelle)
- Versorgung: 3.3V (über Spannungsregler)
- Anschlüsse:
- VCC → 3.3V
- GND → gemeinsame Masse
- SDA → Pin 20 (I²C)
- SCL → Pin 21 (I²C)
- PCLK, HREF, VSYNC, D0–D7 → an digitale Pins (z. B. 2–9)

- XCLK → an Pin 10 (über Taktgenerator)

4. Bewegungsmodule (Motoren)

- Motoren: 2 DC-Motoren
- Treiber: L298N oder TB6612FNG
- Versorgung:
- Logikstrom: 5V (von Arduino)
- Motorstrom: 12V Akku (extern)
- Anschlüsse:
- Motor A → OUT1, OUT2 (Treiber)
- Motor B → OUT3, OUT4 (Treiber)
- Enable → PWM-Pin (z. B. 9, 10)
- IN1–IN4 → digitale Pins (z. B. 2, 3, 4, 5)
- GND → gemeinsame Masse (zwischen Logik und Motorstrom)

5. LEDs (Lichteffekte)

- Typ: WS2812B oder einfache LEDs
- Versorgung: 5V (über Spannungsregler)
- Anschlüsse:
- Daten → Pin 6 (z. B. für WS2812B)
- VCC → 5V
- GND → gemeinsame Masse

6. Sensoren

a) Bewegungssensor (PIR)

- Typ: HC-SR501
- Versorgung: 5V
- Anschlüsse:
- VCC → 5V
- GND → gemeinsame Masse
- OUT → digitaler Pin (z. B. 7)

b) Taster

- Typ: mechanische Taster
- Anschlüsse:
- Taster 1 → digitaler Pin (z. B. 8)
- Taster 2 → digitaler Pin (z. B. 11)

- GND → gemeinsame Masse

7. Spannungsversorgung

- Hauptstromversorgung: 12V LiPo-Akku
- Spannungsregler:
 - LM7805 → 5V für Logik (Arduino, Sensoren, LEDs)
 - LM317 → 3.3V für OLED, Kamera
- Gemeinsame Masse: Alle Module (Motor, Logik, Sensoren, Display) teilen eine gemeinsame GND-Leitung

Wokwi-Schaltplan

Ein Wokwi-Schaltplan wird als wokwi/diagram.json erzeugt, um die elektrische Verkabelung visuell darzustellen. Die Module sind wie folgt in Wokwi konfiguriert:

- Arduino Mega (ATmega2560)
- OLED Display (SSD1306)
- Kamera (OV7670)
- L298N Motor-Treiber
- WS2812B LEDs
- PIR-Sensor
- Taster
- Spannungsregler (LM7805, LM317)

Wichtige Hinweise

- Gemeinsame Masse: Alle Module (Motor, Logik, Sensoren, Display) müssen über eine gemeinsame GND-Leitung verbunden sein.
- Motorstrom: Der Motorstrom wird über eine externe 12V-Versorgung geliefert, die vom Arduino getrennt ist. Die GND-Leitungen müssen dennoch miteinander verbunden sein.
- Pegelprüfung: Die Logikmodule (z. B. Arduino) arbeiten mit 5V, Sensoren (z. B. Kamera) benötigen 3.3V. Es ist wichtig, Spannungsregler und Pegelwandler zu verwenden.
- Datenblätter: Für alle Module sollten die Datenblätter herangezogen werden, um die korrekten Spannungen, Stromwerte und Pinbelegungen zu überprüfen.

Zusammenfassung

Die Hardware des Hund-Roboters ist modular aufgebaut und über I²C und SPI an den Arduino Mega angeschlossen. Die Stromversorgung ist getrennt in Logik- und Motorstrom aufgeteilt, wobei eine gemeinsame Masse für alle Module verwendet wird. Die Module sind in Wokwi als wokwi/diagram.json dargestellt und dienen als Grundlage für die Projektplanung und Simulation.

Dieses Wiring-Dokument dient als didaktische Anleitung zur elektrischen Verkabelung und soll als Grundlage für die Entwicklung eines funktionsfähigen Roboters dienen.

Hinweis: Hinweis: Dieser Inhalt wurde lokal mit RoboForge Local erzeugt. Er ist eine Lern- und Entwurfsvorlage und muss vor realer Hardware, Druck oder Abgabe geprüft werden.

Wokwi-Schaltungsprofil

```
{  
  
  "project_family": "legged_robot",  
  
  "controller": "arduino_mega",  
  
  "components": [  
    {  
  
      "kind": "servo",  
  
      "count": 8,  
  
      "label": "Bein-Servos für 4 Beine; Servo-Aktuator"  
    },  
    {  
  
      "kind": "mpu6050",  
  
      "count": 1,  
  
      "label": "Lagesensor; IMU"  
    },  
    {  
  
      "kind": "dht22",  
  
      "count": 1,  
  
      "label": "Temperatur/Feuchte"  
    },  
    {  
  
      "kind": "oled",  
  
      "count": 1,  
  
      "label": "I2C-Anzeige"  
    },  
    {  
  
      "kind": "pir",  
  
      "count": 1,  
  
      "label": "PIR-Bewegungssensor"  
    },  
    {  
  
      "kind": "ldr",  
  
      "count": 1,  
  
      "label": "Lichtsensorm"  
    },  
    {  
  
      "kind": "led",  
  
      "count": 1,  
  
      "label": "Status-LED"  
    },  
    {  
  
      "kind": "button",  
  
      "count": 1,  
  
      "label": "Benutzereingabe"  
    }  
  ],  
}
```

```
"power": "external_5v_servo_power",  
  "notes": []  
}
```

Code-Skelett

/

Projekt: Hund-Roboter

Zweck: Grundgerüst für einen beweglichen, lichtstarken Haustier-ähnlichen Roboter auf Basis von Arduino Mega.

Warnung: Bitte prüfe den Pinout und die Hardware vor dem Upload!

Hardwareanforderungen:

- 2 DC-Motoren mit Treiber (z. B. L298N)
- 8 LEDs (z. B. als Beleuchtung)
- OLED-Display (SSD1306, I²C)
- PIR-Bewegungssensor
- Taster für manuelle Steuerung
- Kamera (OV7670) - TODO: Implementierung fehlt
- WLAN-Modul (ESP32) - TODO: Implementierung fehlt

Funktionen:

- Bewegung (vorwärts, rückwärts, links/rechts)
- Lichteffekte (LEDs)
- OLED-Display als „Gesicht“
- Eingabeverarbeitung (Taster, Sensor)

/

```
#include <Wire.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_GFX.h>
```

```
// Pin-Definitionen
```

```
#define MOTOR_LEFT_A 2
```

```
#define MOTOR_LEFT_B 3
```

```
#define MOTOR_RIGHT_A 4
```

```
#define MOTOR_RIGHT_B 5
```

```
#define LED_PIN 6
```

```
#define PIR_PIN 7
```

```
#define BUTTON_PIN 8
```

```
// OLED Display
```

```
#define SCREEN_WIDTH 128
```

```
#define SCREEN_HEIGHT 64
```

```
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
```

```
// Globale Variablen
bool motorRunning = false;
bool ledState = false;
int buttonState = 0;
int lastButtonState = 0;
```

```
void setup() {
```

```
  Serial.begin(9600);
```

```

setupHardware();

printStatus();
}

void loop() {

readInputs();

updateMotors();

updateDisplay();

delay(100);
}

void setupHardware() {
  // Motorpins als Ausgang

pinMode(MOTOR_LEFT_A, OUTPUT);
pinMode(MOTOR_LEFT_B, OUTPUT);
pinMode(MOTOR_RIGHT_A, OUTPUT);
pinMode(MOTOR_RIGHT_B, OUTPUT);

  // LED und Taster

pinMode(LED_PIN, OUTPUT);
pinMode(BUTTON_PIN, INPUT);
pinMode(PIR_PIN, INPUT);

  // OLED Display initialisieren
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {

Serial.println(F("OLED-Display konnte nicht initialisiert werden"));

    for (;;) { // Fehler, Programm stoppt
    }

display.clearDisplay();

display.setTextSize(1);

display.setTextColor(SSD1306_WHITE);

display.setCursor(0, 0);

display.println("Hund-Roboter");

display.display();
}

void readInputs() {
  // Taster-Eingabe
  buttonState = digitalRead(BUTTON_PIN);
  if (buttonState != lastButtonState) {
    if (buttonState == HIGH) {

motorRunning = !motorRunning; // Toggle Motor

    }
  }
  lastButtonState = buttonState;

  // PIR-Sensor
  if (digitalRead(PIR_PIN) == HIGH) {
    ledState = true;

digitalWrite(LED_PIN, HIGH);

  } else {
    ledState = false;

digitalWrite(LED_PIN, LOW);

```

```

    }
}

void updateMotors() {
    if (motorRunning) {
        // Vorwärts fahren

digitalWrite(MOTOR_LEFT_A, HIGH);
digitalWrite(MOTOR_LEFT_B, LOW);
digitalWrite(MOTOR_RIGHT_A, HIGH);
digitalWrite(MOTOR_RIGHT_B, LOW);
    } else {

stopMotors();
    }
}

void stopMotors() {

digitalWrite(MOTOR_LEFT_A, LOW);
digitalWrite(MOTOR_LEFT_B, LOW);
digitalWrite(MOTOR_RIGHT_A, LOW);
digitalWrite(MOTOR_RIGHT_B, LOW);
}

void updateDisplay() {

display.clearDisplay();
display.setCursor(0, 0);
display.println("Hund-Roboter");
display.print("Motor: ");
display.println(motorRunning ? "AN" : "AUS");
display.print("LED: ");
display.println(ledState ? "AN" : "AUS");
display.print("PIR: ");
display.println(digitalRead(PIR_PIN) == HIGH ? "Bewegung" : "Still");
display.display();
}

void printStatus() {

Serial.println("Hund-Roboter gestartet");

Serial.println("Motor: " + String(motorRunning ? "AN" : "AUS"));

Serial.println("LED: " + String(ledState ? "AN" : "AUS"));
}

// TODO: Kamera-Implementierung (OV7670)
// TODO: WLAN-Implementierung (ESP32)

```

Hinweis: Hinweis: Dieser Inhalt wurde lokal mit RoboForge Local erzeugt. Er ist eine Lern- und Entwurfsvorlage und muss vor realer Hardware, Druck oder Abgabe geprüft werden.

Druckteileliste

Druckteileliste

Plattform: legged_robot

Controller: arduino_mega

Automatisch geprüfte Stückzahlen

1x base_plate

Zweck: Hauptbasis des Roboters

Prüfen: Maße prüfen.

1x top_cover

Zweck: Obere Abdeckung; Deckplatte für Elektronik und Kabel.

Prüfen: Maße prüfen.

1x battery_tray

Zweck: Tray für Batterien; Akkuhalter mit Gurtöffnungen.

Prüfen: Maße prüfen.

1x sensor_bracket

Zweck: Halterung für Sensoren

Prüfen: Maße prüfen.

1x hc_sr04_front_mount

Zweck: Mount für Ultraschallsensor

Prüfen: Maße prüfen.

1x camera_mount

Zweck: Mount für Kamera; Kamerahalter.

Prüfen: Maße prüfen.

1x display_bezel

Zweck: Rand für OLED-Display; Displayrahmen.

Prüfen: Maße prüfen.

10x standoffs

Zweck: Abstandshalter; Abstandshalter-Sets.

Prüfen: Maße prüfen.

1x controller_tray

Zweck: Tray für Arduino Mega; Halter für den gewählten Controller.

Prüfen: Maße prüfen.

1x led_strip_clip

Zweck: Befestigung für LED-Streifen

Prüfen: Maße prüfen.

6x cable_clip_set

Zweck: Kabelbefestigung

Prüfen: Maße prüfen.

2x screw_fit_test

Zweck: Prüfung der Schraubenpassung; Teststück für Schraubenpassung.

Prüfen: Maße prüfen.

1x legged_body_plate

Zweck: Zentrale Körperplatte für 4 Beine.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

1x servo_power_distribution_plate

Zweck: Verteilerplatte für externe Servoversorgung.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

8x standard_servo_u_bracket

Zweck: Servo-U-Halter für Hüfte/Schulter/Knie.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

4x leg_link_long

Zweck: Obere Beinsegmente.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

4x leg_link_short

Zweck: Untere Beinsegmente.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

4x foot_pad

Zweck: Druckbare Füße für jedes Bein.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

1x small_sensor_clip

Zweck: Sensorclip.

Prüfen: Maße, Bohrungen und Druckausrichtung prüfen.

Sanity-Check

- Unpassende Druckteile entfernt: caster_mount, motor_driver_tray, n20_motor_mount, wheel_hub.
- top_cover: doppelte Einträge auf 1 reduziert.

- battery_tray: doppelte Einträge auf 1 reduziert.
- sensor_bracket: doppelte Einträge auf 1 reduziert.
- camera_mount: doppelte Einträge auf 1 reduziert.
- display_bezel: doppelte Einträge auf 1 reduziert.
- controller_tray: doppelte Einträge auf 1 reduziert.
- led_strip_clip: doppelte Einträge auf 1 reduziert.

Wichtig

Diese Liste ist eine Lern- und Entwurfsvorlage. Vor dem Druck müssen echte Komponenten, Schrauben, Servoabmessungen, Kabelwege und Wandstärken geprüft werden.

3D-Druckvorlage

```
// RoboForge Local - universelle komponentenbasierte OpenSCAD-Teilebibliothek
// Deutsche finale Version v7.1.
// Dieses Modell ist eine Lernvorlage, keine produktionsfertige CAD-Datei.
// Ziel: mehr echte mechanische Einzelteile statt einer dekorativen Szene.
// Reale Platinen-, Akku-, Motor-, Servo- und Schraubenmaße vor dem Druck prüfen.
```

\$fn = 48;

```
preview_colors = false;
fit_clearance = 0.4;
m3_clearance = 3.2;
plate_thickness = 4;
```

```
module maybe_color(name) {
  if (preview_colors) {
    if (name == "plate") color([0.6,0.6,0.6]) children();
    else if (name == "tray") color([0.2,0.5,0.8]) children();
    else if (name == "bracket") color([0.8,0.5,0.2]) children();
    else if (name == "arm") color([0.7,0.7,0.2]) children();
    else if (name == "test") color([0.3,0.7,0.3]) children();
    else children();
  } else children();
}
```

```
module label2d(txt, size=5) {
```

```
  linear_extrude(height=0.6) text(txt, size=size, halign="center", valign="center");
```

```
}
```

```
module rounded_plate(l, w, h, r=6) {
```

```
  maybe_color("plate") linear_extrude(height=h) hull() {
```

```
    translate([r, r]) circle(r=r);
    translate([l-r, r]) circle(r=r);
    translate([r, w-r]) circle(r=r);
    translate([l-r, w-r]) circle(r=r);
  }
```

```
}
```

```
module m3_hole(x, y, h=10) {
  translate([x, y, -0.1]) cylinder(h=h, d=m3_clearance);
}
```

```
module slot(x, y, l, w, h=10) {
  translate([x, y, -0.1]) hull() {
    cylinder(h=h, d=w);
    translate([l,0,0]) cylinder(h=h, d=w);
  }
}
```

```
module mounting_grid(l, w, h, step=20) {
  for (x=[20:step:l-20]) for (y=[20:step:w-20]) m3_hole(x, y, h+0.2);
}
```

```
module tray_box(l=60, w=35, wall=3, h=8, label="TRAY") {
```

```
  maybe_color("tray") difference() {
```

```
    union() {
      cube([l + wall*2, w + wall*2, h]);
      translate([(l+wall*2)/2, -3, h]) label2d(label, 5);
    }
    translate([wall, wall, 2]) cube([l, w, h+0.1]);
  }
```

```
  m3_hole(8, 8, h+1); m3_hole(l+wall*2-8, 8, h+1);
```

```
  m3_hole(8, w+wall*2-8, h+1); m3_hole(l+wall*2-8, w+wall*2-8, h+1);
```

```
}
```

```
module base_plate(l=120, w=90, h=4) {
  difference() {
```

```
    rounded_plate(l, w, h);
```

```
    mounting_grid(l, w, h);
```

```
    slot(l/2-18, w/2, 36, 5, h+0.2);
```

```
    slot(12, w/2, 18, 4, h+0.2);
```

```

slot(l-30, w/2, 18, 4, h+0.2);
}
}

module robot_arm_base_plate(l=145, w=110, h=4) {
    difference() {

rounded_plate(l, w, h, 7);

mounting_grid(l, w, h, 18);

        translate([l-55, w/2-18, -0.1]) cylinder(h=h+0.2, d=42);

for(a=[0:90:270]) translate([l-55 + cos(a)30, w/2 + sin(a)30, -0.1]) cylinder(h=h+0.2, d=m3_clearance);

slot(18, 18, 35, 5, h+0.2);

slot(18, w-18, 35, 5, h+0.2);

    }
}

module x_frame_plate(l=130, h=4) {

maybe_color("plate") difference() {

    union() {
        translate([0, l/2-10, 0]) cube([l,20,h]);
        translate([l/2-10,0,0]) cube([20,l,h]);
        translate([l/2,l/2,0]) cylinder(h=h, d=36);
    }
    translate([l/2,l/2,-0.1]) cylinder(h=h+0.2, d=18);

for(a=[45:90:315]) translate([l/2+cos(a)45,l/2+sin(a)45,-0.1]) cylinder(h=h+0.2,d=m3_clearance);

    }
}

module top_cover(l=110, w=75, h=3) {
    difference() {

rounded_plate(l, w, h, 5);

m3_hole(12, 12, h+0.2); m3_hole(l-12, 12, h+0.2);

m3_hole(12, w-12, h+0.2); m3_hole(l-12, w-12, h+0.2);

slot(l/2-22, w/2, 44, 8, h+0.2);

    }
}

module controller_tray(controller="GEN") {
    if (controller == "arduino_uno") tray_box(72, 54, 3, 9, "ARDUINO");
    else if (controller == "arduino_nano") tray_box(48, 20, 3, 8, "NANO");
    else if (controller == "arduino_mega") tray_box(102, 54, 3, 9, "MEGA");
    else if (controller == "raspberry_pi_pico") tray_box(55, 24, 3, 8, "PICO");
    else if (controller == "esp32" || controller == "esp32_cam") tray_box(58, 32, 3,
        8, "ESP32");
    else tray_box(64, 38, 3, 8, "CTRL");
}

module motor_driver_tray() { tray_box(52, 36, 3, 8, "DRIVER"); }
module stepper_driver_tray() { tray_box(42, 24, 3, 8, "A4988"); }
module relay_module_tray() { tray_box(50, 35, 3, 8, "RELAY"); }

module battery_tray(l=72, w=42, h=12) {

maybe_color("tray") difference() {

    union() { cube([l+6, w+6, h]); translate([(l+6)/2, -3, h]) label2d("AKKU",5);
    }
    translate([3, 3, 2]) cube([l, w, h+0.1]);

slot(8, 0, 14, 5, h+0.2); slot(l-14, w+6, 14, 5, h+0.2);

    }
}

module battery_18650_holder() {

maybe_color("tray") difference() {

    cube([84, 24, 14]);

```

```

    translate([9,12,5]) rotate([0,90,0]) cylinder(h=66, d=18.6);
    translate([8, -0.1, 5]) cube([12, 24.2, 7]);

m3_hole(5,5,15); m3_hole(79,19,15);
}
}

module battery_strap_plate() {

maybe_color("plate") difference() {

rounded_plate(80, 42, 4, 5);

slot(8,10,64,5,5); slot(8,30,64,5,5);

m3_hole(8,8,5); m3_hole(72,34,5);

}
}

module motor_clamp() {

maybe_color("bracket") difference() {

    cube([42, 22, 18]);
    translate([5, -0.1, 5]) cube([32, 22.2, 9]);

m3_hole(7, 11, 19); m3_hole(35, 11, 19);

}
}

module n20_motor_mount() {

maybe_color("bracket") difference() {

    union() { cube([38,18,20]); translate([0,18,0]) cube([38,8,4]); }
    translate([8,-0.1,7]) cube([22,18.2,10]);
    translate([19,-0.1,12]) rotate([-90,0,0]) cylinder(h=18.4,d=12);

m3_hole(6,22,5); m3_hole(32,22,5);

}
}

module caster_mount() {

maybe_color("bracket") difference() {

rounded_plate(42,32,5,5);

    translate([21,16,-0.1]) cylinder(h=5.2,d=18);

m3_hole(8,8,6); m3_hole(34,8,6); m3_hole(8,24,6); m3_hole(34,24,6);

}
}

module sensor_bracket() {

maybe_color("bracket") difference() {

    union() {
        cube([55, 8, 38]);
        translate([0, 0, 30]) cube([55, 20, 8]);
        translate([5, 8, 0]) cube([8, 12, 30]);
        translate([42, 8, 0]) cube([8, 12, 30]);
    }
    translate([14, -0.1, 33]) rotate([-90,0,0]) cylinder(h=22, d=16);
    translate([41, -0.1, 33]) rotate([-90,0,0]) cylinder(h=22, d=16);

m3_hole(8, 4, 9); m3_hole(47, 4, 9);

}
}

module hc_sr04_front_mount() {

maybe_color("bracket") difference() {

    union() { cube([60, 6, 34]); translate([0,6,0]) cube([60,12,6]); }
    translate([17,-0.1,22]) rotate([-90,0,0]) cylinder(h=7,d=16.5);
    translate([43,-0.1,22]) rotate([-90,0,0]) cylinder(h=7,d=16.5);

```

```

m3_hole(6,12,7); m3_hole(54,12,7);
}
}

module small_sensor_clip() {

maybe_color("bracket") difference() {
    cube([34,24,10]);
    translate([4,4,2]) cube([26,16,9]);

m3_hole(5,5,11); m3_hole(29,19,11);
}
}

module standoff_set() {
    for (x=[0,18,36,54]) translate([x,0,0]) difference() {
        cylinder(h=18, d=8);
        translate([0,0,-0.1]) cylinder(h=18.2, d=m3_clearance);
    }
}

module wheel_hub() {

maybe_color("bracket") difference() {
    union() { cylinder(h=10, d=36); translate([0,0,10]) cylinder(h=3,d=20); }
    translate([0,0,-0.1]) cylinder(h=13.2, d=5.2);

for(a=[0:60:300]) rotate([0,0,a]) translate([9,-2,-0.1]) cube([9,4,10.2]);
}
}

module sg90_servo_bracket() {
    // 9g/SG90 style cradle. Dimensions are intentionally parametric placeholders.

maybe_color("bracket") difference() {
    union() {
        cube([36, 18, 28]);
        translate([-6,0,0]) cube([48,18,4]);
        translate([0,-4,22]) cube([36,4,6]);
    }
    translate([3, 3, 5]) cube([30, 12, 25]);
    translate([18,-4.1,25]) rotate([-90,0,0]) cylinder(h=5,d=7);

m3_hole(-1,4,5); m3_hole(37,14,5);
}
}

module standard_servo_u_bracket() {
    // MG996R/MG995 class U bracket concept.

maybe_color("bracket") difference() {
    union() {
        cube([52, 12, 38]);
        translate([0,36,0]) cube([52,12,38]);
        translate([0,12,0]) cube([52,24,5]);
    }
    translate([26,-0.1,25]) rotate([-90,0,0]) cylinder(h=48.2,d=8);
    translate([8,6,-0.1]) cylinder(h=5.2,d=m3_clearance);
    translate([44,42,-0.1]) cylinder(h=5.2,d=m3_clearance);
}
}

module rotating_base_servo_mount() {

maybe_color("arm") difference() {
    union() {
        cylinder(h=8, d=76);
        translate([-30,-20,8]) cube([60,40,8]);
        translate([-24,-12,16]) cube([48,24,14]);
    }
    translate([0,0,-0.1]) cylinder(h=30.2, d=8);

for(a=[0:90:270]) rotate([0,0,a]) translate([28,0,-0.1]) cylinder(h=30.2, d=m3_clearance);
    translate([-18,-7,17]) cube([36,14,14.2]);
}
}

```

```

module rotating_base() { rotating_base_servo_mount(); }

module pan_tilt_base() {

maybe_color("arm") difference() {

    union() { rounded_plate(70,55,5,6); translate([18,12,5]) cube([34,31,24]); }
    translate([35,27.5,13]) rotate([0,90,0]) cylinder(h=40,d=8);

m3_hole(8,8,6); m3_hole(62,47,6);

    translate([22,18,6]) cube([26,19,24]);
    }
}

module arm_link(l=105, w=18, h=6) {

maybe_color("arm") difference() {

    hull() {
        translate([w/2,w/2,0]) cylinder(h=h, d=w);
        translate([l-w/2,w/2,0]) cylinder(h=h, d=w);
    }
    translate([w/2,w/2,-0.1]) cylinder(h=h+0.2, d=5.5);
    translate([l-w/2,w/2,-0.1]) cylinder(h=h+0.2, d=5.5);

for(x=[35:25:l-35]) translate([x,w/2,-0.1]) cylinder(h=h+0.2, d=8);

    }
}

module arm_link_short() { arm_link(75,18,6); }
module arm_link_long() { arm_link(118,20,6); }

module leg_link_long() {
    // Oberes Beinsegment für Spinnen-, Hexapod- oder Hundroboter.
    // Eigenes Modul, damit Beinprofile nicht fälschlich Roboterarm-Links verwenden.

maybe_color("arm") difference() {

    hull() {
        translate([11,11,0]) cylinder(h=6, d=22);
        translate([107,11,0]) cylinder(h=6, d=22);
    }
    translate([11,11,-0.1]) cylinder(h=6.2, d=5.6);
    translate([59,11,-0.1]) cylinder(h=6.2, d=5.6);
    translate([107,11,-0.1]) cylinder(h=6.2, d=5.6);
    // Materialfenster zur Gewichtsreduzierung.
    translate([28,5,-0.1]) cube([22,12,6.2]);
    translate([68,5,-0.1]) cube([22,12,6.2]);
    }
}

module leg_link_short() {
    // Unteres Beinsegment für modulare Laufroboter.

maybe_color("arm") difference() {

    hull() {
        translate([10,10,0]) cylinder(h=6, d=20);
        translate([68,10,0]) cylinder(h=6, d=20);
    }
    translate([10,10,-0.1]) cylinder(h=6.2, d=5.6);
    translate([39,10,-0.1]) cylinder(h=6.2, d=5.6);
    translate([68,10,-0.1]) cylinder(h=6.2, d=5.6);
    translate([25,4,-0.1]) cube([28,12,6.2]);
    }
}

module elbow_joint_plate() {

maybe_color("arm") difference() {

rounded_plate(62,38,6,6);

    translate([19,19,-0.1]) cylinder(h=6.2,d=6);
    translate([43,19,-0.1]) cylinder(h=6.2,d=6);

m3_hole(8,8,7); m3_hole(54,30,7);

    }
}

module gripper_palm() {

maybe_color("arm") difference() {

```



```

        union() { cube([54,38,8]); translate([17,-14,0]) cube([20,14,8]); }
        translate([27,19,-0.1]) cylinder(h=8.2,d=8);

m3_hole(8,8,9); m3_hole(46,8,9); m3_hole(8,30,9); m3_hole(46,30,9);
    }
}

module gripper_finger_pair() {

maybe_color("arm") union() {

    difference() {
        translate([0,0,0]) cube([72,10,8]);
        translate([14,-0.1,-0.1]) cube([10,10.2,8.2]);
        translate([50,-0.1,-0.1]) cube([8,10.2,8.2]);

m3_hole(8,5,9); m3_hole(64,5,9);

        }
        difference() {
            translate([0,28,0]) cube([72,10,8]);
            translate([14,27.9,-0.1]) cube([10,10.2,8.2]);
            translate([50,27.9,-0.1]) cube([8,10.2,8.2]);

m3_hole(8,33,9); m3_hole(64,33,9);

        }
    }
}

module servo_horn_adapter() {

maybe_color("arm") difference() {

    union() { cylinder(h=5,d=30); for(a=[0:90:270]) rotate([0,0,a])
        translate([0,-4,0]) cube([38,8,5]); }
    translate([0,0,-0.1]) cylinder(h=5.2,d=5);

for(a=[0:45:315]) rotate([0,0,a]) translate([11,0,-0.1]) cylinder(h=5.2,d=1.8);

    }
}

module servo_power_distribution_plate() {

maybe_color("plate") difference() {

rounded_plate(84,42,4,5);

for(x=[14,28,42,56,70]) { slot(x, 12, 0.1, 3, 5); slot(x, 30, 0.1, 3, 5); }

m3_hole(6,6,5); m3_hole(78,6,5); m3_hole(6,36,5); m3_hole(78,36,5);

    }
}

module camera_mount() {

maybe_color("bracket") difference() {

    union() {
        cube([45, 10, 45]);
        translate([0, 0, 35]) cube([45, 28, 10]);
    }
    translate([22.5, -0.1, 40]) rotate([-90,0,0]) cylinder(h=30, d=18);

m3_hole(8, 5, 12); m3_hole(37, 5, 12);

    }
}

module display_bezel() {

maybe_color("bracket") difference() {

    cube([72, 34, 4]);
    translate([6, 5, -0.1]) cube([60, 24, 4.2]);

m3_hole(4,4,5); m3_hole(68,4,5); m3_hole(4,30,5); m3_hole(68,30,5);

    }
}

module potentiometer_panel() {

```

```

maybe_color("bracket") difference() {
    cube([80, 36, 4]);

for(x=[18,40,62]) translate([x,18,-0.1]) cylinder(h=4.2, d=7.5);

m3_hole(6,6,5); m3_hole(74,30,5);
}
}

module joystick_panel() {

maybe_color("bracket") difference() {
    cube([64, 54, 4]);
    translate([32,27,-0.1]) cylinder(h=4.2, d=28);

m3_hole(7,7,5); m3_hole(57,7,5); m3_hole(7,47,5); m3_hole(57,47,5);
}
}

module universal_l_bracket() {

maybe_color("bracket") difference() {
    union() { cube([50,5,30]); cube([50,28,5]); }

m3_hole(8,14,6); m3_hole(42,14,6);

    translate([12,-0.1,18]) rotate([-90,0,0]) cylinder(h=6,d=m3_clearance);
    translate([38,-0.1,18]) rotate([-90,0,0]) cylinder(h=6,d=m3_clearance);
}
}

module universal_u_bracket() {

maybe_color("bracket") difference() {
    union() { cube([52,6,28]); translate([0,32,0]) cube([52,6,28]);
        translate([0,6,0]) cube([52,26,5]); }
    translate([26,-0.1,17]) rotate([-90,0,0]) cylinder(h=38.2,d=6);

m3_hole(8,18,6); m3_hole(44,18,6);
}
}

module universal_bracket() { universal_u_bracket(); }

module cable_clip_set() {

maybe_color("bracket") for(i=[0:3]) translate([i*18,0,0]) difference() {
    cube([12,16,8]);
    translate([6,8,-0.1]) cylinder(h=8.2,d=7);
    translate([6,16,-0.1]) cube([2,5,8.2], center=true);
}
}

module buzzer_clip() {

maybe_color("bracket") difference() { cube([28,28,8]); translate([14,14,-0.1]) cylinder(h=8.2,d=18);
m3_hole(4,4,9); m3_hole(24,24,9); }
}

module led_strip_clip() {

maybe_color("bracket") difference() { cube([70,12,6]); translate([5,3,2]) cube([60,6,5]); m3_hole(4,6,7);
m3_hole(66,6,7); }
}

module screw_fit_test() {

maybe_color("test") difference() {
    cube([70,18,4]);

for(i=[0:4]) translate([10+i*12,9,-0.1]) cylinder(h=4.2,d=2.8+i*0.2);
}
}

```

```

module sensor_box_bottom(l=115, w=85, h=28) {

maybe_color("plate") difference() {

    cube([l, w, h]);
    translate([3,3,3]) cube([l-6, w-6, h+0.1]);
    translate([l-28, w-0.1, 10]) cube([18, 4, 8]);

m3_hole(10,10,h+0.2); m3_hole(l-10,10,h+0.2); m3_hole(10,w-10,h+0.2); m3_hole(l-10,w-10,h+0.2);

}

}

module legged_body_plate(l=150, w=110, h=4) {
    difference() {

rounded_plate(l, w, h, 10);

mounting_grid(l, w, h, 22);

        for (yy=[18, w-18]) for (xx=[22:28:l-22]) slot(xx-6, yy, 12, 4, h+0.2);

slot(l/2-24, w/2, 48, 7, h+0.2);

    }

}

module foot_pad() {

maybe_color("bracket") difference() {

    union() { rounded_plate(42, 26, 5, 6); translate([7,8,5]) cube([28,10,5]); }

m3_hole(8,13,11); m3_hole(34,13,11);

slot(13,13,16,4,11);

    }

}

module side_servo_rail() {

maybe_color("bracket") difference() {

    cube([130, 18, 8]);
    for (xx=[15:25:115]) { m3_hole(xx, 9, 9); translate([xx-5,3,-0.1])
        cube([10,12,8.2]); }

    }

}

module track_side_plate() {

maybe_color("bracket") difference() {

    hull() { translate([18,18,0]) cylinder(h=5,d=36); translate([110,18,0])
        cylinder(h=5,d=36); }
    translate([18,18,-0.1]) cylinder(h=5.2,d=18);
    translate([110,18,-0.1]) cylinder(h=5.2,d=18);

for(xx=[42:20:88]) m3_hole(xx,18,6);

    }

}

module motor_pod() {

maybe_color("bracket") difference() {

    union() { cube([42,30,10]); translate([21,15,10]) cylinder(h=12,d=24); }
    translate([21,15,-0.1]) cylinder(h=22.2,d=12);

m3_hole(6,6,11); m3_hole(36,24,11);

    }

}

module prop_guard_segment() {

maybe_color("bracket") difference() {

linear_extrude(height=4) difference() { circle(d=72); circle(d=62); translate([-40,-40]) square([80,40]); }

        translate([0,0,-0.1]) cylinder(h=4.2,d=8);

    }

}

```

```

}

module display_stand() {

maybe_color("bracket") difference() {

    union() { cube([14,58,8]); translate([0,44,8]) rotate([25,0,0])
    cube([14,8,42]); }

m3_hole(7,10,9); m3_hole(7,46,9);

    }
}

module weather_louver() {

maybe_color("bracket") union() {

    difference() { cube([78,36,4]); translate([6,6,-0.1]) cube([66,24,4.2]); }
    for (yy=[7:7:28]) translate([8,yy,4]) rotate([20,0,0]) cube([62,3,5]);
    }
}

module gripper_finger() {

maybe_color("arm") difference() {

    union() { cube([76,11,8]); translate([58,0,0]) cube([18,18,8]); }
    translate([14,-0.1,-0.1]) cube([10,11.2,8.2]);
    translate([46,-0.1,-0.1]) cube([8,11.2,8.2]);

m3_hole(8,5.5,9); m3_hole(66,9,9);

    }
}

// Print plate: Einzelteile liegen getrennt auf der XY-Ebene.
// Platform: legged_robot
// Controller: arduino_mega
// Generated modules: base_plate, top_cover, battery_tray, sensor_bracket,
hc_sr04_front_mount, camera_mount, display_bezel, standoffs, standoffs, standoffs,
standoffs, standoffs, standoffs, standoffs, standoffs, standoffs,
controller_tray, led_strip_clip, cable_clip_set, cable_clip_set, cable_clip_set,
cable_clip_set, cable_clip_set, cable_clip_set, screw_fit_test, screw_fit_test,
legged_body_plate, servo_power_distribution_plate, standard_servo_u_bracket,
standard_servo_u_bracket, standard_servo_u_bracket, standard_servo_u_bracket,
standard_servo_u_bracket, standard_servo_u_bracket, standard_servo_u_bracket,
standard_servo_u_bracket, leg_link_long, leg_link_long, leg_link_long,
leg_link_long, leg_link_short, leg_link_short, leg_link_short, leg_link_short,
foot_pad, foot_pad, foot_pad, foot_pad, small_sensor_clip

// Part 1: base plate
translate([0, 0, 0]) base_plate(200, 150, 8);

// Part 2: top cover
translate([224, 0, 0]) top_cover(180, 130, 3);

// Part 3: battery tray
translate([0, 174, 0]) battery_tray(72, 42, 12);

// Part 4: sensor bracket
translate([108, 174, 0]) sensor_bracket();

// Part 5: hc sr04 front mount
translate([192, 174, 0]) hc_sr04_front_mount();

// Part 6: camera mount
translate([280, 174, 0]) camera_mount();

// Part 7: display bezel
translate([354, 174, 0]) display_bezel();

// Part 8: standoffs
translate([0, 252, 0]) standoff_set();

// Part 9: standoffs
translate([90, 252, 0]) standoff_set();

// Part 10: standoffs
translate([180, 252, 0]) standoff_set();

// Part 11: standoffs
translate([270, 252, 0]) standoff_set();

// Part 12: standoffs
translate([360, 252, 0]) standoff_set();

// Part 13: standoffs
translate([0, 296, 0]) standoff_set();

```

```

// Part 14: standoffs
translate([90, 296, 0]) standoff_set();

// Part 15: standoffs
translate([180, 296, 0]) standoff_set();

// Part 16: standoffs
translate([270, 296, 0]) standoff_set();

// Part 17: standoffs
translate([360, 296, 0]) standoff_set();

// Part 18: controller tray
translate([0, 340, 0]) controller_tray("arduino_mega");

// Part 19: led strip clip
translate([140, 340, 0]) led_strip_clip();

// Part 20: cable clip set
translate([240, 340, 0]) cable_clip_set();

// Part 21: cable clip set
translate([338, 340, 0]) cable_clip_set();

// Part 22: cable clip set
translate([0, 432, 0]) cable_clip_set();

// Part 23: cable clip set
translate([98, 432, 0]) cable_clip_set();

// Part 24: cable clip set
translate([196, 432, 0]) cable_clip_set();

// Part 25: cable clip set
translate([294, 432, 0]) cable_clip_set();

// Part 26: screw fit test
translate([0, 478, 0]) screw_fit_test();

// Part 27: screw fit test
translate([100, 478, 0]) screw_fit_test();

// Part 28: legged body plate
translate([200, 478, 0]) legged_body_plate(200, 150, 8);

// Part 29: servo power distribution plate
translate([0, 652, 0]) servo_power_distribution_plate();

// Part 30: standard servo u bracket
translate([114, 652, 0]) standard_servo_u_bracket();

// Part 31: standard servo u bracket
translate([196, 652, 0]) standard_servo_u_bracket();

// Part 32: standard servo u bracket
translate([278, 652, 0]) standard_servo_u_bracket();

// Part 33: standard servo u bracket
translate([360, 652, 0]) standard_servo_u_bracket();

// Part 34: standard servo u bracket
translate([0, 728, 0]) standard_servo_u_bracket();

// Part 35: standard servo u bracket
translate([82, 728, 0]) standard_servo_u_bracket();

// Part 36: standard servo u bracket
translate([164, 728, 0]) standard_servo_u_bracket();

// Part 37: standard servo u bracket
translate([246, 728, 0]) standard_servo_u_bracket();

// Part 38: leg link long
translate([0, 804, 0]) leg_link_long();

// Part 39: leg link long
translate([148, 804, 0]) leg_link_long();

// Part 40: leg link long
translate([296, 804, 0]) leg_link_long();

// Part 41: leg link long
translate([0, 856, 0]) leg_link_long();

// Part 42: leg link short
translate([148, 856, 0]) leg_link_short();

// Part 43: leg link short
translate([254, 856, 0]) leg_link_short();

// Part 44: leg link short
translate([0, 908, 0]) leg_link_short();

```

```

// Part 45: leg link short
translate([106, 908, 0]) leg_link_short();

// Part 46: foot pad
translate([212, 908, 0]) foot_pad();

// Part 47: foot pad
translate([284, 908, 0]) foot_pad();

// Part 48: foot pad
translate([356, 908, 0]) foot_pad();

// Part 49: foot pad
translate([0, 964, 0]) foot_pad();

// Part 50: small sensor clip
translate([72, 964, 0]) small_sensor_clip();

// Hinweise:
// - Dies ist eine universelle Druckplatte aus einzelnen Teilen, keine vormontierte
  Szene.
// - Die Teile sind bewusst modular: Halter, Schalen, Laschen, Links, Trays und
  Teststücke.
// - Teile im Slicer bei Bedarf trennen, drehen und mit passenden Druckeinstellungen
  platzieren.
// - Reale Servo-, Motor-, Akku-, Platinen- und Schraubenmaße vor dem Druck prüfen.
// - Profilnotiz: Der Roboter ist auf einer 4-Rad-Plattform aufgebaut. Die Kamera und
  das OLED-Display sind über I²C angeschlossen. Die Lichteffekte werden durch
  LED-Streifen realisiert. Die Bewegung erfolgt über zwei DC-Motoren mit Treiber. Die
  Steuerung erfolgt über Arduino Mega.

```

Abschließende Prüfliste

- Pinout des konkreten Boards prüfen.
- Spannungen, Ströme, Akku und gemeinsame Masse prüfen.
- Wokwi diagram.json als Lernvorlage behandeln.
- OpenSCAD-Datei vor dem Druck mit realen Bauteilmaßen abgleichen.
- Code-Skelett vor echter Hardware anpassen und testen.